

Presentation For PIPELINED RISC-V PROCESSOR ON PYNQ-Z2 FPGA

Saumilkumar savani & Zhenkai Lin

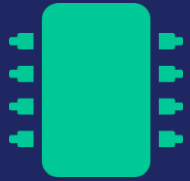
Introduction to HDL presentation

Chair of Computer Architecture and Operating Systems

TUM Campus Heilbronn – TUM School of Computation, Information and Technology (CIT)

Technical University of Munich

06.August.2026

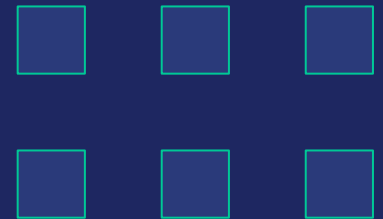


RV32I

Pipelined RISC-V Processor

Design & Verification on the PYNQ-Z2 FPGA

From RTL design to hardware/software co-verification — five pipeline stages, real silicon, a working sort.



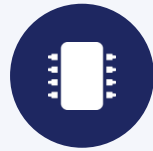
The Objective

Not just RTL on paper — a processor running real software on real hardware.



Design

Build a 5-stage pipelined RISC-V core in RTL, supporting the RV32I instructions needed for sorting.



Integrate

Drop the core into a real FPGA system on the PYNQ-Z2, wired to the ARM processing system.

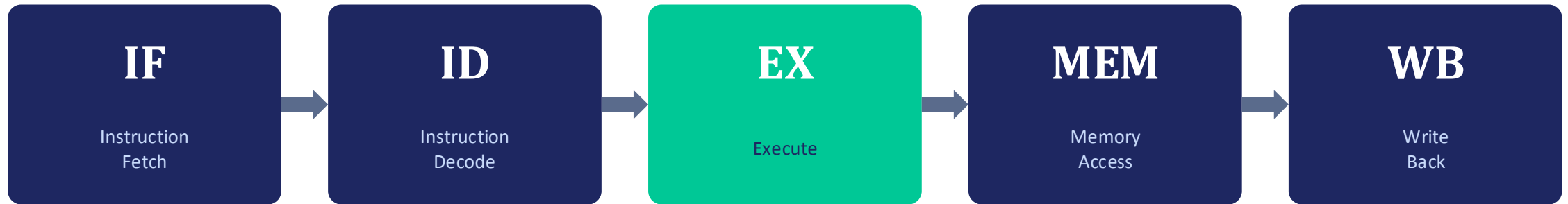


Verify

Prove correctness from software: load a program, run it, and check the result on-chip.

A Five-Stage Pipeline

Every instruction moves through the same five stages, one per clock cycle.



Instruction support: RV32I subset

- Arithmetic & logic operations
- Loads and stores
- Branches and jumps
- LUI — for building large constants (e.g. the DONE address)

Two Halves, One System

The ARM Processing System (PS) drives the RISC-V core sitting in Programmable Logic (PL).



Processing System (PS)

- Loads the bitstream
- Writes the program into instruction BRAM
- Writes input values into data BRAM
- Releases reset through AXI GPIO
- Waits for completion
- Verifies the result



controls



Programmable Logic (PL)

The RISC-V core lives here, alongside two separate BRAMs:

Instruction BRAM

Written by PS, read by the core — fetch happens independently of data access.

Data BRAM

Shared by PS and core — no structural hazard between fetch and data access.

Memory Map

Four addresses tie the whole system together.

Instruction BRAM

0x40000000

Program storage

Reset GPIO

0x41200000

AXI GPIO reset control

Data BRAM

0x42000000

Inputs, working data, results



DONE Flag

Stored at data-memory offset **0x1000**. When the sort completes, the program writes the magic value **0xCAFEBADE** to this location — the PS's signal to check the result.

A Layered Verification Approach

Each step builds confidence before the next — no jumping straight to “run the program.”

- 1 Check that the bitstream, hardware handoff file, and program HEX file exist
- 2 Load the FPGA overlay
- 3 Verify AXI addresses and create the MMIO objects
- 4 Write/readback test both BRAMs
- 5 Inject the program into instruction BRAM and read it back word by word

Keeping the Pipeline Honest

Pipelining means instructions overlap — hazards have to be handled, not ignored.



Forwarding

A dependent instruction can receive a value directly from the Memory or Write-Back stage — without waiting for it to land in the register file.

Result: no stall needed for most data dependencies.



Load-Use Stall

When an instruction needs a value that a load hasn't fetched yet, Fetch and Decode stall until the loaded data becomes available.

Result: correctness is protected — no instruction ever reads a stale value.

Verified on Real Hardware

A signed-integer sort, run on the board, checked against a software reference.

Negative values

Duplicates

Zero

Positive values

2×

sorting tests passed on the board

reverse-order & signed-integer sorts



matched the Python software
reference

final BRAM contents compared directly

1×

DONE write, no writes after

0xCAFEBADE, exactly once, then an infinite JAL loop

What It Costs on the Chip

Post-route resource usage and timing for the complete pipelined design.

10,189

LUTs used

19.15% of available

11,626

Flip-Flops used

10.93% of available

6.5

Block-RAM tiles

0 DSP blocks used



50 MHz timing constraint met

Positive worst negative slack (WNS) of 7.081 ns on the routed design.

Bonus: Pipelined vs. Single-Cycle

Same FPGA, same program, same BRAM setup, same DONE protocol — a fair, apples-to-apples comparison.

Single-Cycle Core

- Simpler design — no pipeline registers, forwarding muxes, hazard detection, or flush logic
- Fewer registers, simpler control
- Critical path spans fetch → decode → ALU → memory → write-back in one cycle
- Longer critical path limits maximum clock frequency

Pipelined Core

- Work is split across five stages — each with less combinational logic
- Needs more registers and extra control logic
- Usually reaches a higher clock frequency
- Load-use hazards and taken branches cause stalls or flushes, so CPI isn't always exactly 1

Compared on: clock frequency · cycle count · CPI · execution time · IPC · post-route LUT / FF / BRAM usage



Conclusion

- A complete PS–PL hardware/software workflow, on real PYNQ-Z2 hardware
- Verified end-to-end — from bitstream loading to correct sorted output
- Pipeline hazards handled correctly: forwarding and load-use stalls
- 50 MHz timing target met, with room to spare (WNS +7.081 ns)
- Bonus single-cycle core sets up a fair pipelined-vs-single-cycle comparison

Thank you

QUESTIONS?

Thank you so much!

